

✓ Module 07 Lab - Better Model Evaluation

Objective: To move beyond simple accuracy and learn how to use more sophisticated and reliable evaluation techniques, including the confusion matrix, precision, recall, and cross-validation.

In this lab, you will write the code to generate and interpret these advanced evaluation metrics.

✓ Part 1: Setup and Model Training

Let's first train a model so we have something to evaluate. We will use the Titanic dataset again to predict survival.

```
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.linear_model import LogisticRegression
4
5 # Load and prepare data
6 df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/dataset
7 df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
8
9 # Fill missing Age values with the median age
10 # Reassignment is used instead of inplace=True, which is deprecated in curr
11 df['Age'] = df['Age'].fillna(df['Age'].median())
12
13 features = ['Age', 'Pclass', 'Sex', 'Fare']
14 target = 'Survived'
15
16 X = df[features]
17 y = df[target]
18
19 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ra
20
21 # Train a model
22 model = LogisticRegression(max_iter=1000)
23 model.fit(X_train, y_train)
24
25 y_pred = model.predict(X_test)
```

✓ Part 2: The Confusion Matrix

Concept: A confusion matrix gives you a more detailed breakdown of a model's performance than accuracy alone. It's a table that shows you where your model got things right and where it got them wrong.

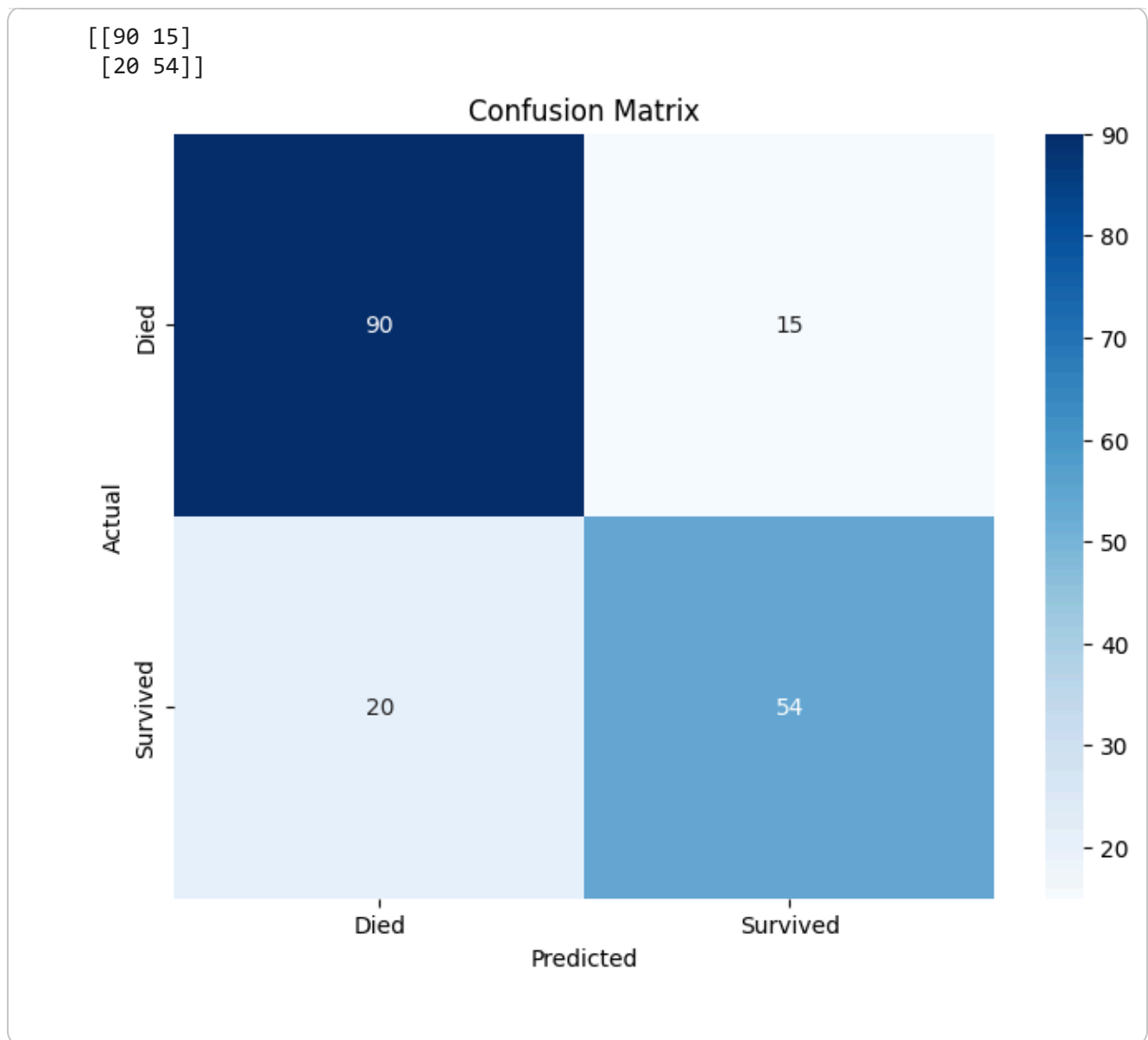
It has four quadrants:

- **True Positives (TP):** Correctly predicted positive (e.g., predicted survival, and they did survive).
- **True Negatives (TN):** Correctly predicted negative (e.g., predicted did not survive, and they did not).
- **False Positives (FP):** Incorrectly predicted positive (e.g., predicted survival, but they did not). Also called a "Type I Error".
- **False Negatives (FN):** Incorrectly predicted negative (e.g., predicted did not survive, but they did). Also called a "Type II Error".

✓ Task 1: Generate and Plot a Confusion Matrix

Your Task: Use `confusion_matrix` from `sklearn.metrics` to calculate the matrix and `seaborn.heatmap` to visualize it.

```
1 from sklearn.metrics import confusion_matrix
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4
5 # --- ENTER YOUR CODE HERE ---
6
7 # 1. Calculate the confusion matrix by comparing the actual test labels to
8 cm = confusion_matrix(y_test, y_pred)
9 print(cm)
10
11 # 2. Create a heatmap so the four quadrants are easy to read
12 #     annot=True prints the counts, fmt='d' keeps them as whole numbers
13 plt.figure(figsize=(8, 6))
14 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
15             xticklabels=['Died', 'Survived'],
16             yticklabels=['Died', 'Survived'])
17 plt.xlabel('Predicted')
18 plt.ylabel('Actual')
19 plt.title('Confusion Matrix')
20 plt.show()
```



✓ Part 3: Precision, Recall, and F1-Score

Concept: From the confusion matrix, we can calculate more nuanced metrics:

- **Precision:** Of all the times the model predicted **positive**, how often was it correct?
 - Formula: $\frac{TP}{TP + FP}$
 - *Use Case:* When the cost of a **False Positive** is high. (e.g., a spam filter; you don't want to incorrectly mark an important email as spam).
- **Recall (Sensitivity):** Of all the actual **positives**, how many did the model correctly identify?
 - Formula: $\frac{TP}{TP + FN}$
 - *Use Case:* When the cost of a **False Negative** is high. (e.g., a medical test for a serious disease; you don't want to miss a real case).

- **F1-Score:** The harmonic mean of Precision and Recall. It provides a single score that balances both.

✓ Task 2: Generate a Classification Report

Your Task: Use `classification_report` from `sklearn.metrics` to get a summary of these metrics for each class.

```
1 from sklearn.metrics import classification_report
2
3 # --- ENTER YOUR CODE HERE ---
4
5 # 1. Generate precision, recall, and F1 score for each class
6 #     target_names replaces the 0 and 1 labels with readable class names
7 report = classification_report(y_test, y_pred, target_names=['Died', 'Survived'])
8 print(report)
```

	precision	recall	f1-score	support
Died	0.82	0.86	0.84	105
Survived	0.78	0.73	0.76	74
accuracy			0.80	179
macro avg	0.80	0.79	0.80	179
weighted avg	0.80	0.80	0.80	179

✓ Part 4: Cross-Validation

Concept: A single train-test split can be lucky or unlucky. What if, by chance, all the "easy" examples ended up in our test set? Our accuracy score would be misleadingly high.

Cross-Validation (CV) solves this. It splits the data into multiple "folds" (e.g., 5 or 10). It then trains and evaluates the model multiple times, using a different fold as the test set each time. The final score is the average of the scores from all folds.

This gives a much more robust and reliable estimate of the model's true performance.

✓ Task 3: Perform Cross-Validation

Your Task: Use `cross_val_score` from `sklearn.model_selection` to perform 5-fold cross-validation on your model.

```
1 from sklearn.model_selection import cross_val_score
2
3 # --- ENTER YOUR CODE HERE ---
4
5 # 1. Perform 5-fold cross-validation
6 # The entire dataset (X, y) is passed in because cross_val_score handles
7 cv_scores = cross_val_score(model, X, y, cv=5)
8
9 # 2. Print the score for each fold, the average, and the spread across folds
10 print(f"Scores for each fold: {cv_scores}")
11 print(f"Average CV Score: {cv_scores.mean():.2%}")
12 print(f"Standard Deviation of CV Scores: {cv_scores.std():.4f}")
```

```
Scores for each fold: [0.7877095  0.78089888 0.78651685 0.7752809  0.80337079]
Average CV Score: 78.68%
Standard Deviation of CV Scores: 0.0094
```



Knowledge Check

Instructions: Answer the following questions in this markdown cell.

1. **Describe a real-world scenario where you would care more about a model's Precision than its Recall.**
2. **Describe a real-world scenario where you would care more about a model's Recall than its Precision.**
3. **Why is a cross-validation score generally more trustworthy than a score from a single train-test split?**

Question 1: Scenario Prioritizing Precision

In a credit card fraud detection system where fraud is the positive class, precision is far more critical than recall because a false positive carries a much higher real-world cost than a false negative. A false positive means a legitimate transaction is falsely flagged as fraudulent, causing a customer's card to be frozen mid purchase leaving them stranded unable to buy gas or check into a hotel while waiting on hold with bank support. On the other hand, a false negative merely results in an unauthorized charge posting to the account, which the customer can easily dispute and have reversed by the bank. In our evaluation, 15 out of 179 cases were false positives; prioritizing precision minimizes these severe customer friction points, ensuring legitimate transactions go through smoothly.

Question 2: Scenario Prioritizing Recall

In predictive machinery maintenance, where "part is about to fail" is the positive class, recall is far more critical than precision because a false negative is drastically more costly than a false positive. A false negative means the model predicts a part is healthy

when it is actually about to fail, leading to catastrophic machine breakdowns, unplanned downtime, and severe operational expense. By contrast, a false positive simply causes technicians to replace a healthy part early, which is a minor, low-cost preventive measure. With our model achieving a recall of 0.73 on the positive class, roughly a quarter of impending real failures are still being missed, highlighting why maximizing recall is essential to prevent costly equipment failures.

Question 3: Why Cross-Validation is More Trustworthy

A single train-test split yielded an accuracy of 80.00%, whereas 5-fold cross validation revealed an average score of 78.68% with individual fold scores ranging from 77.53% to 80.34%. This gap demonstrates that a single split can be overly optimistic or misleading due to a "lucky" random distribution of data in the test set. Cross validation is much more trustworthy because it tests the model across multiple diverse subsets, proving that the true generalized performance fluctuates around 78.68% rather than consistently hitting the peak 80.00% single split result.